

Text as Data for Economic Research

Representing Meaning

Class 3. Word and Document Embeddings

June 9, 2026

Tiziano Rotesi

University of Turin

Where This Sits in the Course

- **Class 1:** we built measures by hand — dictionaries, counts, discriminating words, PMI seed-expansion.
- **Class 2:** we let the computer *learn* (supervised) or *discover* (unsupervised) the categories.
- **Today:** we change the *representation* of words themselves — from isolated dimensions to a geometry of meaning.

Embeddings are a tool that feeds back into every task we have seen: better dictionaries, better similarity, better clustering.

From Counts to Meaning

Up to Now: Words as One-Hot Vectors

In the document-term matrix, each word is its own dimension:

$$cat = (0, 0, 0, 1, 0, 0, 0, 0)$$

$$dog = (0, 0, 0, 0, 0, 0, 1, 0)$$

$$car = (0, 0, 1, 0, 0, 0, 0, 0)$$

Up to Now: Words as One-Hot Vectors

In the document-term matrix, each word is its own dimension:

$$cat = (0, 0, 0, 1, 0, 0, 0, 0)$$

$$dog = (0, 0, 0, 0, 0, 0, 1, 0)$$

$$car = (0, 0, 1, 0, 0, 0, 0, 0)$$

These vectors are mutually **orthogonal** — every word is equidistant from every other.

cat and *dog* are exactly as dissimilar as *cat* and *car*. The representation carries **no notion of similarity**.

What Counts Still Cannot Do

Cosine on the DTM rewards documents for using *the same words*.

- “The central bank raised rates” and “The Fed hiked interest” share almost no tokens — cosine ≈ 0 .
- Synonymy, paraphrase, and related concepts are invisible.

We want a representation where *rates* and *interest* are close **because they mean similar things**, not because they are spelled the same.

The Distributional Hypothesis

The **context** in which a word appears is a clue to its **meaning**.

“You shall know a word by the company it keeps.”

(Firth, 1957)

Operational idea: words that appear in similar contexts should get similar vectors. We will turn this slogan into an estimable model.

From Sparse Counts to Dense Embeddings

An **embedding** is a distributed representation of a word:

- reduces dimension from the vocabulary size J (tens of thousands) to a **dense** $K \approx 100-300$;
- every coordinate is active — meaning is spread across the vector, not one hot entry;
- proximity in this space is designed to track **semantic similarity**.

The same compression idea as PCA/LSA in Class 2 — but learned to predict context, not to reconstruct counts.

Word2Vec

Learning Embeddings as Self-Supervision

We have no labels — but we observe, for free, *which words occur near which*.

- Turn the corpus into a **prediction problem**: predict a word from its context (or vice versa).
- “Self-supervised”: the labels are manufactured from the text itself.
- Train on a large general corpus; optionally refine on your own.

Familiar logic for economists: we are about to run a giant penalized logistic regression.

Word2Vec: Two Architectures

An umbrella term for two complementary objectives (Mikolov et al. 2013):

CBOW — predict the focal word from its context:

the quick brown [...] jumps over the lazy dog

Skip-gram — predict the context from the focal word:

the [...] [...] fox [...] [...] the lazy dog

Skip-gram is the more common default; we develop it below.

Skip-gram: the Setup

... the $\overbrace{\text{quick brown}}^{\text{Context}}$ $\underbrace{\text{fox}}_{\text{Target}}$ $\overbrace{\text{jumps over}}^{\text{Context}}$ the lazy dog ...

How to retrieve the embeddings, in four steps:

1. treat the target and its context neighbors as **positive** examples;
2. randomly sample other words as **negative** examples;
3. train a logistic classifier to tell the two apart;
4. keep the learned weights — those *are* the embeddings.

The Classifier: Notation

- w — the target word; c — a candidate context word
- $\mathbf{w}, \mathbf{c}$ — their embedding vectors
- $\mathbf{W}, \mathbf{C}$ — the target and context embedding *matrices*
- $P(+ \mid w, c)$ — probability c truly is a context word for w
- $P(- \mid w, c) = 1 - P(+ \mid w, c)$

Two matrices: each word has one vector as a target, one as context.

The Classifier: Probability

Similarity between target and context is their **dot product**:

$$\text{similarity}(w, c) \approx \mathbf{c} \cdot \mathbf{w}$$

To express this similarity as a probability, use the logistic function — exactly the logistic regression of Class 2:

$$P(+ \mid w, c) = \sigma(\mathbf{c} \cdot \mathbf{w}) = \frac{1}{1 + \exp(-\mathbf{c} \cdot \mathbf{w})}$$

Context words should score high; random words should score low.

Negative Sampling

For target fox with a context window of 1:

- **Positive** (+): (fox, brown), (fox, jumps)
- **Negative** (−), sampled at random: (fox, egg), (fox, process), (fox, cup), (fox, silver)

Loss for this target (negative log-likelihood):

$$L(\mathbf{W}, \mathbf{C}) = -\log \left[P(+ \mid \text{fox, brown}) P(+ \mid \text{fox, jumps}) \right. \\ \cdot P(- \mid \text{fox, egg}) P(- \mid \text{fox, process}) \\ \left. \cdot P(- \mid \text{fox, cup}) P(- \mid \text{fox, silver}) \right]$$

Negative sampling avoids the costly softmax over the whole vocabulary.

The Objective, Over the Whole Corpus

The example above is for one target word; we generalize it to the whole corpus.

Algorithm (sketch):

- . Initialize the word matrix $\mathbf{W}$ and context matrix $\mathbf{C}$.
- 1. For each target word, treat nearby words as positives and sampled unrelated words as negatives.
- 2. Evaluate the loss and update $\mathbf{W}$ and $\mathbf{C}$ by stochastic gradient descent.
- 3. Repeat over many passes through the corpus until the model converges.

Each update uses the same $(\sigma(\mathbf{c} \cdot \mathbf{w}) - y)$ gradient as the logistic regression of Class 2 — just applied to manufactured labels.

What the Model Learns

- Words appearing in **similar contexts** are pushed toward **similar vectors** — the distributional hypothesis, realized.
- Each word ends with two vectors ($\mathbf{w}$ as target, $\mathbf{c}$ as context): average them, or just keep $\mathbf{w}$.

The embedding matrix $\mathbf{W}$ is the deliverable — a dense vector per word, ready to use downstream.

The Context Window Is a Choice

The window size controls *what kind* of similarity you learn:

- **Small window** (± 2): tight, substitutable words.
- **Large window** (± 5 or more): broad topical relatedness.

Levy & Goldberg (2014): with ± 2 , the nearest neighbors of “Hogwarts” are other fictional schools; with ± 5 , they are Harry Potter characters (Dumbledore, Malfoy). **Same word, different geometry.**

Beyond Word2Vec: Other Static Embeddings

GloVe

Factorizes a global word–word co-occurrence matrix. Co-occurring words get similar vectors; trains fast on aggregated counts.

fastText

Represents words as bags of character n -grams (subwords). Handles morphology and **out-of-vocabulary** words — useful for noisy or inflected corpora.

Doc2Vec extends the idea to whole documents. *Static* embeddings give one vector per word; **contextual** embeddings (Class 4) give a different vector per occurrence.

From Word Vectors to a Document Vector

Word2Vec gives a vector *per word*. To represent a whole (short) text, the standard baseline is to **aggregate its word vectors**:

- **Average** (or sum) the token embeddings — one vector per document. The centroid is the document; averaging keeps it length-robust.
- **Weight** the average — by TF-IDF, or SIF (Arora et al. 2017: inverse-frequency weights, minus the common component) — a famously hard-to-beat sentence baseline.

Strong for **short** text (headlines, tweets), where little word order is lost. **But** it ignores order and **dilutes** meaning over long documents — the case for *trained* document embeddings next.

Notebook: spaCy's `doc.vector` is exactly this token-average — our baseline against MiniLM.

Modern Embeddings You Can Download

Beyond single-word vectors, you can download **sentence / document embedding** models: they map a *whole passage* to one dense vector, trained so that paraphrases land close — no averaging of word vectors by hand.

Representative Hugging Face models:

- `sentence-transformers/all-MiniLM-L6-v2`: small, fast classroom baseline;
- `Qwen/Qwen3-Embedding-0.6B`: newer, stronger, heavier family.

These are **transformer-based** — we open that box in Class 4. Model choice is empirical: benchmark broadly (e.g. MTEB), then validate on your own task.

Notebook: we use `all-MiniLM-L6-v2` on newspaper headlines — era validation, semantic search, and reprint detection.

The Geometry of Meaning

Cosine Similarity, Now on Words

The same vector-space formula from Class 2 — applied to dense word vectors:

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}$$

Now $\cos(\text{rates}, \text{interest})$ is **high** even though the words never share a spelling — because they share *company*.

This is the single most-used operation on embeddings: nearest neighbors, concept scores, analogy.

Similarity vs. Relatedness

Embeddings blur two distinct relations — worth keeping straight:

Semantic similarity

same kind of thing

- synonyms: *car* ~ *automobile*
- category: *car* ~ *vehicle*

Semantic relatedness

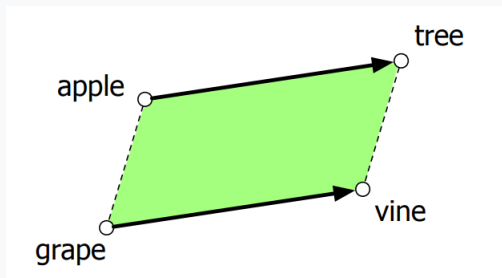
associated, not alike

- function: *car* ~ *driving*
- part-whole: *car* ~ *tire*
- place: *car* ~ *road*

A high cosine does not tell you *which* — read the neighbors.

The Parallelogram: Analogies

Embeddings can encode *relational* meaning: “ a is to b as a^* is to ?”



$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}},$$

$$\mathbf{v}_{\text{Paris}} - \mathbf{v}_{\text{France}} + \mathbf{v}_{\text{Italy}} \approx \mathbf{v}_{\text{Rome}}$$

Analogies Are Suggestive, Not Magic

The parallelogram works best for clean, frequent relations (capitals, gender of nouns). It is fragile elsewhere:

- the “answer” is usually the nearest neighbor *excluding* the three input words — a built-in nudge;
- rare words and abstract relations break it.

Treat analogies as a **diagnostic** of the embedding space, not a measurement instrument.

Pitfall: the Black-Sheep Problem

Embeddings capture what is *said*, not what is *true*.

- Obvious properties go unstated: most sheep are white, so “white sheep” is rare in text.
- Result: word2vec may rate $\text{sim}(\text{black}, \text{sheep}) > \text{sim}(\text{white}, \text{sheep})$.

Matters for economists measuring beliefs or attitudes from text: the corpus reports the marked, surprising, or contested — not the default.

Pitfall: Antonyms and Word Senses

- **Antonyms look similar.** *love* and *hate* share contexts (“I ____ this movie”), so they sit close — opposite meaning, similar vector.
- **Senses are merged.** Static embeddings give *one* vector for “like” (verb) and “like” (preposition), or “bank” (river/finance).
- **Phrases break.** “New York” $\neq$ “new” + “york” — tokenize multiword expressions before training.

Contextual models (Class 4) resolve senses; static embeddings do not.

Pretrained vs. Custom

Two Roads to an Embedding

Pretrained (GloVe, Google News word2vec, spaCy)

Trained on very large general corpora; broad, stable, free. spaCy's medium/large English models include 300-dimensional GloVe-style vectors. A strong default.

Custom (train on your own corpus)

Captures domain- and era-specific meaning (19th-century newspapers, central-bank speak), but needs enough text and is noisier.

“Fed” means something different in a finance corpus than on the open web — that is the case for going custom.

Rodriguez & Spirling (2022)

Word Embeddings

What works, what doesn't, and how to tell the difference
for applied research

The Applied-Research Question

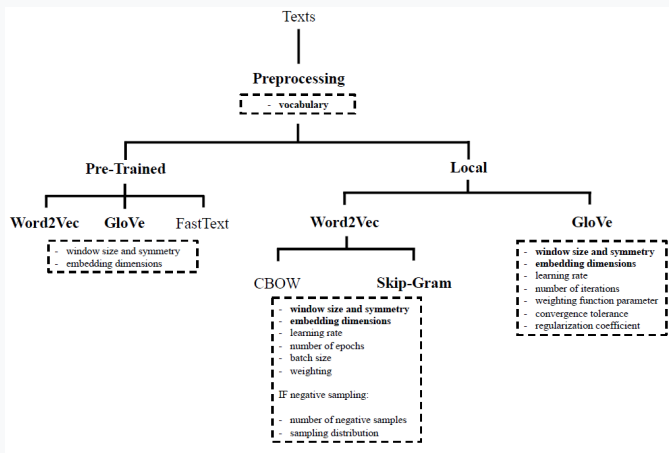
How sensitive are embedding results to the choices we make?

- local vs. pretrained;
- word2vec vs. GloVe;
- window size $\in \{1, 6, 12, 24, 48\}$, dimensions $\in \{50, 100, 200, 300, 450\}$.

They evaluate two tasks on the *Congressional Record*: political attention (topic-like) and how parties use words differently (supervised-like).

How They Judge Quality: a Turing Test

Compare **machine** nearest-neighbors against **human**-generated ones for anchor terms (*democracy, freedom, immigration, taxes, ...*).



A concrete, human-grounded **validation** of the embedding space — the recurring theme, applied to representation itself.

Practical Advice (in Their Setting)

- Very small dimensions or context windows can underperform; larger choices add cost without always changing conclusions.
- **Pretrained embeddings generally perform well** — go custom mainly when specialized meaning is essential.
- Rare words can swing a locally trained model: mind the vocabulary.

The exact choice is corpus-specific — the *procedure* (validate against humans, test robustness) is what generalizes.

Applications

What to Use Embeddings For?

- **Downstream tasks for words:** better representations for clustering, classification, and retrieval.
- **Downstream tasks for documents:** aggregate word vectors, or use a document embedding, to compare short texts by meaning.
- **Semantics:** use distances and directions in the embedding space as measurements of meaning, association, and change.

Same warning as always: embeddings create measures, and measures need validation.

Comparing Meaning: Skills

Hansen, Ramdas, Sadun & Fuller (2021), *The Demand for Executive Skills*

- Research goal: measure which skills firms demand in executive searches.
- Build six skill clusters from O*NET descriptions for chief executives, then score search documents by similarity to those clusters.
- Function: a domain-specific embedding model turns text similarity into a generated measure of demand for cognitive, operational, and interpersonal skills.

Expanding Concept Sets: Corporate Culture

Li, Mai, Shen & Yan (2021), *Measuring Corporate Culture*

- Start with five seed values from corporate-culture work: *innovation, integrity, quality, respect, teamwork*.
- Train word2vec on earnings-call transcripts; use nearby words and phrases to expand each culture dictionary.
- Score firm-years from Q&A language, then validate against external markers and simpler text measures.

Expanding Concept Sets: FOMC Uncertainty

Cieslak, Hansen, McMahon & Xiao (2024), *Policymakers' Uncertainty*

- Research goal: measure uncertainty perceived in private FOMC deliberations and connect it to policy stance.
- Embedding use: begin with seed terms such as *uncertain*, *uncertainty*, *risk*, and *risks*; use CBOW embeddings trained on FOMC transcripts to expand the uncertainty vocabulary.
- Function: embeddings expand the uncertainty side of a topic-specific dictionary; the final measure comes from co-occurrence with inflation, real-economy, market, and model-uncertainty terms.

Document-to-Concept Scores: Emotion

Gennaro & Ash (2022), *Emotion and Reason in Political Language*

- Research goal: measure emotionality in U.S. congressional speeches.
- Embedding use: build an emotion–reason dimension from validated word lists, then compare each speech vector to the two poles.
- Function: embeddings turn a pair of dictionaries into a continuous document score, then the paper validates the score against human judgment.

Document-to-Concept Scores: Ideas

Ash, Chen & Naidu (2026), *Ideas Have Consequences*

- Research goal: measure the diffusion of law-and-economics ideas in judicial language and link it to judicial decisions.
- Embedding use: use a law-and-economics lexicon to define a semantic dimension in vector space, then score legal texts along that dimension.
- Function: embeddings make an intellectual influence observable as variation in language, rather than as exact mentions of a few phrases.

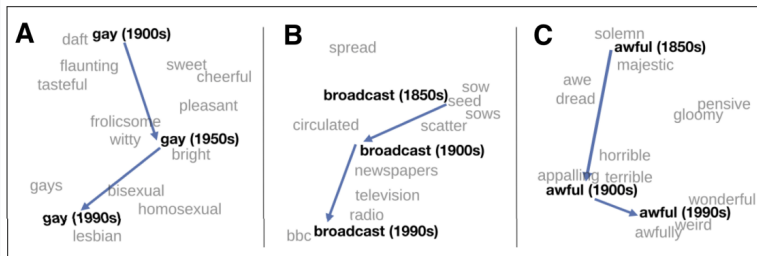
How Concepts Are Related

Kozlowski, Taddy & Evans (2019), *The Geometry of Culture*

- Research goal: study shared cultural meanings of class over the twentieth century.
- Embedding use: define cultural dimensions from word differences, such as *rich-poor*, and project words onto those axes.
- Function: embeddings measure relations among concepts, with survey validation used to check that axes correspond to human-rated associations.

Meaning Over Time

Hamilton, Leskovec & Jurafsky (2016)



- Train comparable embeddings by period and evaluate them against known semantic shifts.
- Function: semantic change becomes an outcome to measure; frequency and polysemy help explain which words move more.

Notebook example: 1858-vs-1958 American Stories.

Cautions

- **Bias:** embeddings learn regularities in the training text, including social stereotypes.
- **Instability:** nearest neighbors can change across random seeds, subsamples, and hyperparameters.
- **Validation:** Rodriguez & Spirling (2022) show why applied researchers should test whether an embedding model works for the specific corpus and task.
- **Generated variables:** embedding-derived measures carry estimation error into downstream regressions.

Takeaways

1. Embeddings replace isolated dimensions with a **geometry of meaning** — learned from context (the distributional hypothesis).
2. Word2Vec is a penalized logistic regression in disguise; cosine and the parallelogram read off the geometry.
3. Use them for similarity, concept expansion, document-to-concept scores, concept relations, and **semantic change**.
4. In applied papers, the same object — a vector — supports different measurement functions.
5. They carry **bias** and **estimation error** — validate, stress-test, and treat the output as a generated variable.

Live: Class 3 notebook. **Practice:** Problem Set 3.